

MARAN PMS — Prompt 44: Planning UX

Contexto

Archivo principal: `client/src/pages/planning.tsx` **Backend relevante:**
`storage.getPlanningData()` **en** `server/storage.ts` **Tipos:** `PlanningCellStatus` ,
`PlanningData` **en** `shared/schema.ts`

El `planning.tsx` no está en el archivo concatenado — buscarlo directamente en Replit
bajo `client/src/pages/planning.tsx`.

CAMBIOS A IMPLEMENTAR

1. Header colapsable

Agregar un botón “▲ Minimizar panel” / “▼ Expandir panel” en la parte superior derecha del planning.

```
const [headerCollapsed, setHeaderCollapsed] = useState(false);
```

Cuando `headerCollapsed === true`:

- Ocultar con `hidden` (o `max-h-0 overflow-hidden transition-all`): filtros de canal/fuente, leyenda de colores, sección de bloqueos sin asignar, cualquier estadística o resumen del período.
- Mantener visible siempre: selector de fechas (date range picker), flechas de navegación anterior/siguiente, botón “Ir a fecha”, y el propio botón de colapsar/expandir.
- El botón debe quedar fijo arriba a la derecha con un ícono `ChevronUp` / `ChevronDown` de `lucide-react`.

El objetivo es que colapsado el header ocupe menos de 48px de altura total, dejando casi toda la pantalla para la grilla.

2. Scroll completo de la página

Problema actual: la parte superior (header del planning) queda fija mientras solo scrollea la grilla de habitaciones.

Fix: cambiar el layout del componente raíz del planning para que TODO el contenido (header + grilla) sea parte de un único flujo scrolleable vertical.

- Quitar cualquier `position: sticky`, `position: fixed`, o `overflow: hidden` del contenedor padre del planning que impida el scroll natural de la página.
- El sidebar del sistema ya es sticky por su propia cuenta — no tocarlo.
- La columna izquierda de números de habitación dentro de la grilla SÍ puede seguir siendo `sticky left-0` para que se vea al hacer scroll horizontal.
- La fila de fechas/días en el header de la tabla SÍ puede seguir siendo `sticky top-0` para que se vea al hacer scroll vertical — pero el header del planning (con filtros, leyenda, etc.) debe scrollear junto con la página.

3. Leyenda de colores — selección por concepto

Problema actual: la leyenda muestra colores sin definición clara y no es personalizable.

Solución:

3a. Definir un mapa de colores fijo por concepto de estado. Reemplazar cualquier lógica de color dispersa por esta única fuente de verdad:

```
export const PLANNING_COLORS: Record<PlanningCellStatus, { bg: string; text: string }> = {
  available: { bg: "bg-white dark:bg-zinc-900", text: "text-zinc-40" },
  booked: { bg: "bg-blue-100 dark:bg-blue-900/40", text: "text-blue-80" },
  checkin_today: { bg: "bg-emerald-400 dark:bg-emerald-600", text: "text-white", },
  checked_in: { bg: "bg-emerald-200 dark:bg-emerald-800", text: "text-emerald-800", },
  checkout_today: { bg: "bg-amber-300 dark:bg-amber-600", text: "text-amber-900", },
  maintenance: { bg: "bg-red-200 dark:bg-red-900/50", text: "text-red-800", },
  cleaning: { bg: "bg-yellow-100 dark:bg-yellow-900/40", text: "text-yellow-800", },
  dirty: { bg: "bg-orange-100 dark:bg-orange-900/40", text: "text-orange-800", },
  group_blocked: { bg: "bg-violet-200 dark:bg-violet-800/50", text: "text-violet-800", },
  early_blocked: { bg: "bg-sky-100 dark:bg-sky-900/30", text: "text-sky-800", },
  late_blocked: { bg: "bg-pink-100 dark:bg-pink-900/30", text: "text-pink-800", },
};
```

3b. La leyenda en el header del planning debe renderizarse a partir de `PLANNING_COLORS`,

mostrando solo los estados que tengan al menos 1 celda en el rango visible. Usar `Object.entries(PLANNING_COLORS)` filtrando por los statuses presentes en `occupancy`.

3c. En el futuro (no implementar ahora) se podrá persistir la paleta en `billing_config` o `system_users`. Por ahora los colores son fijos como arriba.

4. Check-in / Check-out cambian el color automáticamente

Comportamiento esperado: cuando se hace check-in de una reserva, su celda en el planning pasa a `checked_in` (verde). Cuando se hace check-out, la celda pasa a `available` o `dirty` según corresponda.

Fix: después de ejecutar el check-in o check-out (cualquier mutación de `PATCH /api/reservations/:id/checkin` o similar), invalidar la query `["/api/planning"]`:

```
queryClient.invalidateQueries({ predicate: q =>
  Array.isArray(q.queryKey) && q.queryKey[0] === "/api/planning"
});
```

Si ya existe esta invalidación, verificar que funcione correctamente y que el re-fetch actualice los colores de las celdas afectadas sin recargar toda la página.

5. Estado suciedad/limpieza en columna de habitaciones — no bloquear reservas

Problema actual: cuando `room.status === "dirty"`, el backend marca la celda como "dirty" y bloquea la creación de reservas en esa fecha.

Cambios requeridos:

5a. **Backend** — en `getPlanningData()` en `server/storage.ts`, modificar la lógica:

```
// ANTES:
if (room.status === "dirty" && day === todayStr) {
  occupancy[room.id].push("dirty");
  continue; // ← este continue bloquea toda la celda
}

// DESPUÉS:
// "dirty" y "cleaning" NO bloquean la celda. Se registran por separado.
// Solo "maintenance" bloquea.
```

El status `dirty / cleaning` de una habitación debe mostrarse visualmente en la columna de habitaciones (columna fija izquierda), NO en las celdas de fechas. Las celdas de fechas para `dirty / cleaning` deben mostrarse como `available` (o con la reserva que corresponda si la hay).

5b. Frontend — en la columna izquierda de habitaciones (donde aparece el número de hab.), agregar un badge o ícono pequeño que indique el estado:

- `dirty` → ícono `AlertCircle` naranja o badge "Sucia" en naranja
- `cleaning` → ícono `RefreshCw` amarillo o badge "Limpieza" en amarillo
- `inspected` → ícono `CheckCircle` verde claro o badge "Inspeccionada" en verde
- `maintenance` → ícono `Wrench` rojo o badge "Mant." en rojo
- `available` → sin badge (estado normal)

Nota: agregar `"inspected"` a `PlanningCellStatus` en `shared/schema.ts`. Se trata igual que `available` (no bloquea reservas).

5c. El estado `maintenance` Sí sigue bloqueando las celdas (no se pueden crear reservas). `dirty`, `cleaning` e `inspected` NO bloquean.

6. Camaje editable en columna de habitaciones

En la columna fija izquierda del planning, junto al número de habitación, mostrar el tipo de camaje actual con un selector inline editable.

```
// Cada fila de habitación muestra:
<div className="flex flex-col gap-0.5">
  <span className="font-semibold text-sm">{room.roomNumber}</span>
  <Select
    value={room.bedTypeId ?? ""}
    onValueChange={ (val) => updateBedTypeMutation.mutate({ roomId: room.id, bedTy
  >
    <SelectTrigger className="h-5 text-xs border-0 p-0 bg-transparent w-full">
      <SelectValue placeholder="Sin camaje" />
    </SelectTrigger>
    <SelectContent>
      {bedTypes.map(bt => (
        <SelectItem key={bt.id} value={bt.id}>{bt.name}</SelectItem>
      ))}
    </SelectContent>
  </Select>
```

</div>

- Usar `useQuery` para traer los `bedTypes` desde `/api/bed-types` (ya existe el endpoint).
 - La mutation de actualización debe hacer `PATCH /api/rooms/:id` con `{ bedTypeId }`.
 - Si el endpoint no existe como PATCH parcial, usar el PUT existente pasando los demás campos sin cambio.
-

7. Mantenimiento bloquea reservas — suciedad no

Ya cubierto en punto 5. Verificar además que al hacer clic en una celda con status "maintenance" NO abra el dialog de nueva reserva, y muestre en su lugar un tooltip o mensaje "Habitación en mantenimiento".

8. Grilla posicionable desde el día anterior a hoy

Problema: el planning no permite posicionarse antes de la fecha actual, pero operativamente el 13/03 se están procesando check-outs de la noche del 12/03.

Fix: cambiar el `minDate` o validación de navegación para permitir retroceder al menos 1 día antes de hoy. Específicamente: el límite mínimo de la fecha de inicio de la vista debe ser `subDays(today, 1)` en lugar de `today`.

9. Fix "Ir a la fecha" — errores de fecha

Localizar el input/datepicker de "Ir a la fecha" en el planning. El bug reportado es que marca fechas con errores (posiblemente timezone offset).

Fix estándar de timezone para Argentina:

```
// NO usar: new Date(dateString) — interpreta como UTC
// Sí usar:
function parseLocalDate(dateStr: string): Date {
  const [y, m, d] = dateStr.split("-").map(Number);
  return new Date(y, m - 1, d); // sin timezone, local
}
```

Aplicar `parseLocalDate` en el handler del "Ir a la fecha" al convertir el string del input a

objeto Date antes de usarlo para setear el estado de la vista.

10. Filtros de visualización en el planning

Agregar un panel de filtros colapsable (dentro del header colapsable del punto 1) con las siguientes opciones:

```
type PlanningFilter = {
  showEmpty: boolean;      // mostrar habitaciones sin reservas en el rango
  showOccupied: boolean;   // mostrar habitaciones con reservas
  roomTypeIds: string[];   // filtrar por categoría ([] = todas)
  floorFilter: string;     // piso (" " = todos)
  statusFilter: PlanningCellStatus | ""; // filtrar por estado (" " = todos)
};

const [filters, setFilters] = useState<PlanningFilter>({
  showEmpty: true,
  showOccupied: true,
  roomTypeIds: [],
  floorFilter: " ",
});
```

Antes de renderizar las filas de habitaciones, aplicar los filtros:

```
const visibleRooms = rooms.filter(room => {
  // Filtro por categoría
  if (filters.roomTypeIds.length > 0 && !filters.roomTypeIds.includes(room.roomTy
  // Filtro por piso
  if (filters.floorFilter && room.floor !== filters.floorFilter) return false;
  // Filtro por ocupación en el rango visible
  const roomOccupancy = occupancy[room.id] ?? [];
  const hasReservation = roomOccupancy.some(s => s !== "available" && s !== "dirt
  if (!filters.showEmpty && !hasReservation) return false;
  if (!filters.showOccupied && hasReservation) return false;
  // Filtro por estado: si alguna celda del rango tiene el status pedido
  if (filters.statusFilter) {
    const roomOcc = occupancy[room.id] ?? [];
    if (!roomOcc.includes(filters.statusFilter as PlanningCellStatus)) return fal
  }
  return true;
});
```

La UI del filtro: chips/checkboxes pequeños en el header para "Con reservas" / "Sin reservas", un Select de categoría (usando `roomTypes` ya disponibles), y un Select de

piso (derivado de `rooms.map(r => r.floor)` sin duplicados).

11. Tooltip/hover sobre reservas

Al hacer hover sobre una barra de reserva en la grilla, mostrar un tooltip con resumen rápido:

```
// Contenido del tooltip:
// - Nombre del huésped
// - Fechas check-in → check-out (cantidad de noches)
// - Empresa asociada (si tiene)
// - Saldo pendiente (si tiene cargos sin pagar)
// - Estado actual (badge con color de PLANNING_COLORS)
```

Usar el componente `Tooltip` de `shadcn/ui` (`@/components/ui/tooltip`). El tooltip debe aparecer al hover con un delay de ~300ms para no interferir con el click normal.

Los datos del huésped y empresa están disponibles en el objeto

`reservations[reservationId]` que devuelve `/api/planning` . El saldo se puede calcular con un `fetch lazy` a `/api/reservations/:id/balance` solo cuando el tooltip se abre, o agregarlo al objeto de `planning` en el backend.

NOTAS TÉCNICAS

- `PlanningCellStatus` está en `shared/schema.ts` — al agregar/modificar tipos, actualizar allí.
- `getPlanningData` está en `server/storage.ts` — los cambios de backend del punto 5 van allí.
- El componente `planning` usa `useQuery` con `queryKey` `["/api/planning"]` — todas las invalidaciones deben usar ese key exacto.
- `bedTypes` se obtiene de `/api/bed-types` — verificar que este endpoint devuelva array de `{ id, name, code }`.
- Los pisos en Maran van del 1 al 12 — el `Select` de piso puede mostrarlos como strings "1" a "12".
- **No modificar** el comportamiento de las celdas de grupos (`group_blocked`) ni de `early/late check-in` (`early_blocked` , `late_blocked`).

